

Documentação do trabalho de implementação

1. Aplicação escolhida

O projeto consiste em uma aplicação para visualização do fractal de Mandelbrot (e sua generalização, o Multibrot, $Z^n + C$). Para cada ponto C do plano complexo, conta-se quantas iterações são necessárias até o módulo de Z ultrapassar 2 (o ponto escapa) ou até um número máximo de iterações (o ponto é considerado parte do conjunto); esse número é convertido em cor. O usuário navega com o teclado (mover, zoom), altera o expoente em tempo real, gira a paleta de cores e alterna entre três modos de coloração. Como o cálculo de um novo quadro pode ser custoso, ele roda em uma thread separada da interface, que mostra uma animação de carregamento sem travar a janela.

2. Divisão de responsabilidades entre as linguagens

C (interface e apresentação): janela e laço de renderização via raylib (60 FPS); captura de teclado (navegação, expoente, cores, modos de coloração); conversão do resultado numérico de cada pixel em cor (três formulas HSV diferentes, uma por modo); disparo do cálculo de cada quadro em uma thread separada; inicialização e encerramento do interpretador Lua embutido no processo.

Lua (cálculo): concentra exclusivamente a matemática, na função `calcular_frame(width, height, offset_x, offset_y, zoom, power, max_iter)`. Há um caminho otimizado para o Mandelbrot clássico ($\text{power} = 2$, só multiplicações e somas) e um caminho geral para o Multibrot (qualquer outra potência, via forma polar de Z : raiz quadrada, arco-tangente, seno e cosseno). O script não sabe nada sobre janelas, teclado ou cores.

3. Método de integração entre C e Lua

A comunicação acontece via embedding do interpretador Lua dentro do processo C, usando a API oficial do Lua (`lua.h`, `luaXlib.h`, `luaLib.h`). Não há processos separados, o Lua roda como biblioteca no mesmo processo. Dois refinamentos completam o mecanismo:

- Thread separada: a cada mudança de parâmetro, o C cria uma nova thread (`pthread_create`) que executa a função Lua em segundo plano, enquanto a thread principal continua desenhando a interface e só sincroniza no final (`pthread_join`). Apenas uma thread aciona o Lua por vez, evitando acesso concorrente ao mesmo `lua_State`.
- Transporte em bloco (string binária + `memcpy`): em vez de devolver cada pixel individualmente (o que exigiria uma chamada entre as linguagens por pixel), o Lua empacota o resultado de todos os pixels em uma única string binária com `string.pack("ddd", n, zr, zi)` por pixel e `table.concat`, devolvida de uma vez como retorno da função. O C lê essa string com `lua_tolstring()` e copia os bytes diretamente para o vetor de pixels com um único `memcpy()`. Isso exige que a struct `PixelData` (três campos `double` em sequência) tenha exatamente o mesmo layout de memória do formato empacotado em Lua, os dois lados precisam concordar sobre esse formato binário compartilhado. Não há leitura nem escrita em disco em nenhum momento.

Esse método reduz o número de transições entre as duas linguagens durante o cálculo de um quadro: de centenas de milhares de chamadas (uma por pixel) para uma única chamada Lua e uma única leitura do resultado.